Microsoft Foundry

Azure AI foundation and core concepts for pro-code AI

Projects and endpoints | Model catalog and comparison | Prompting and evaluation | Secure, private agents

Choose models with evidence

Evaluate before you trust

Private by design

CodeSizzler · Microsoft Preferred Partner, Data & AI
www.codesizzler.in

Five Days from Cloud Basics to Production-Ready AI

Each day adds one layer. Every day mixes short concepts, a live demo and a hands-on lab.

DAY 1	DAY 2	DAY 3 · TODAY	DAY 4	DAY 5
Azure & Agent Foundations	**Copilot Studio**	**Microsoft Foundry**	**Data, RAG & Power BI**	**Microsoft Fabric**
Agentic workflows with Microsoft 365 Copilot	Low-code and pro-code conversational AI	Azure AI foundation and core concepts	Grounding AI in trusted data	Enterprise analytics with Copilot
• Azure cloud foundations • How AI agents work • Agent Builder, Notebooks, Cowork, Skills • Chaining agents with a human in the loop	• Agents, workflows and apps • Knowledge, MCP tools, skills • Topics and the standard harness • Evaluate, publish, govern	• Projects, resources, endpoints • Model catalog and comparison • Prompting and evaluation • Secure, private, observable agents	• Data quality for AI • Retrieval-augmented generation • Azure AI Search patterns • Power BI basics and dashboards	• OneLake, lakehouse, shortcuts • Copilot in Fabric and Power BI • Fabric IQ ontology and data agents • Governance and the road ahead

Foundations → Microsoft 365 Copilot → Copilot Studio → Foundry → RAG & Power BI → Fabric

Today: From a Model Deployment to a Secure Agent

Concepts and two hands-on labs in the Foundry portal and VS Code.

01 Foundry basics	02 Models	03 Prompts & evaluation	04 Agents & security
Concepts	**Concepts + Lab 2**	**Concepts + Lab 2**	**Architecture**
• What Foundry is and when to use it • Resources, projects, endpoints • Lab 1: prepare a project	• Model catalog and model cards • Leaderboard: quality, cost, safety • Deploy and compare in the playground	• Prompt engineering patterns • Evaluation wizard and evaluators • Synthetic test data	• Foundry Agent Service and SDKs • Identity, private networking • Agent over SAP and OpenText • Guardrails and monitoring

You need

- An Azure subscription you can create resources in
- Visual Studio Code
- Python 3.13
- Git
- Azure CLI

By the end of today you can pick a model with evidence, not opinion.

01

Microsoft Foundry Basics

The Azure platform for models, agents, evaluations and observability

- What Foundry is
- When to use it
- Resources & projects
- Endpoints & keys

What Is Microsoft Foundry?

One Azure platform to choose models, build agents, evaluate them and watch them in production. Formerly Azure AI Foundry.

01 Model catalog

Deploy and compare OpenAI models, open models and others. Fine-tune where supported.

02 Foundry Agent Service

Managed runtime for pro-code agents with tools such as MCP, AI Search and Fabric data agents.

03 Evaluations

Built-in evaluators for quality, groundedness and safety; synthetic test data; AI red teaming.

04 Observability

Tracing of every model and tool call into Application Insights; monitoring dashboards.

05 Guardrails

Content filters and Prompt Shields against direct and document-based attacks.

06 Developer tools

Portal at ai.azure.com, SDKs for Python and .NET, Foundry Toolkit for VS Code.

Portal: ai.azure.com · Built on the same Azure identity, networking and monitoring you met on Day 1.

When Foundry Is the Right Choice

Foundry means you own the infrastructure, security, network and on-call. Choose it for a reason.

Use Foundry when

A requirement forces code.

- Custom orchestration or multi-agent code
- A specific model, or fine-tuning
- Private access to SAP, OpenText or plant data
- Automated evaluation in CI/CD and red teaming
- The agent is part of an app your developers own

Don't use it when

A simpler platform is enough.

- A maker could build it in Copilot Studio in a week
- It is a personal assistant over M365 files
- It only answers numbers from Fabric tables

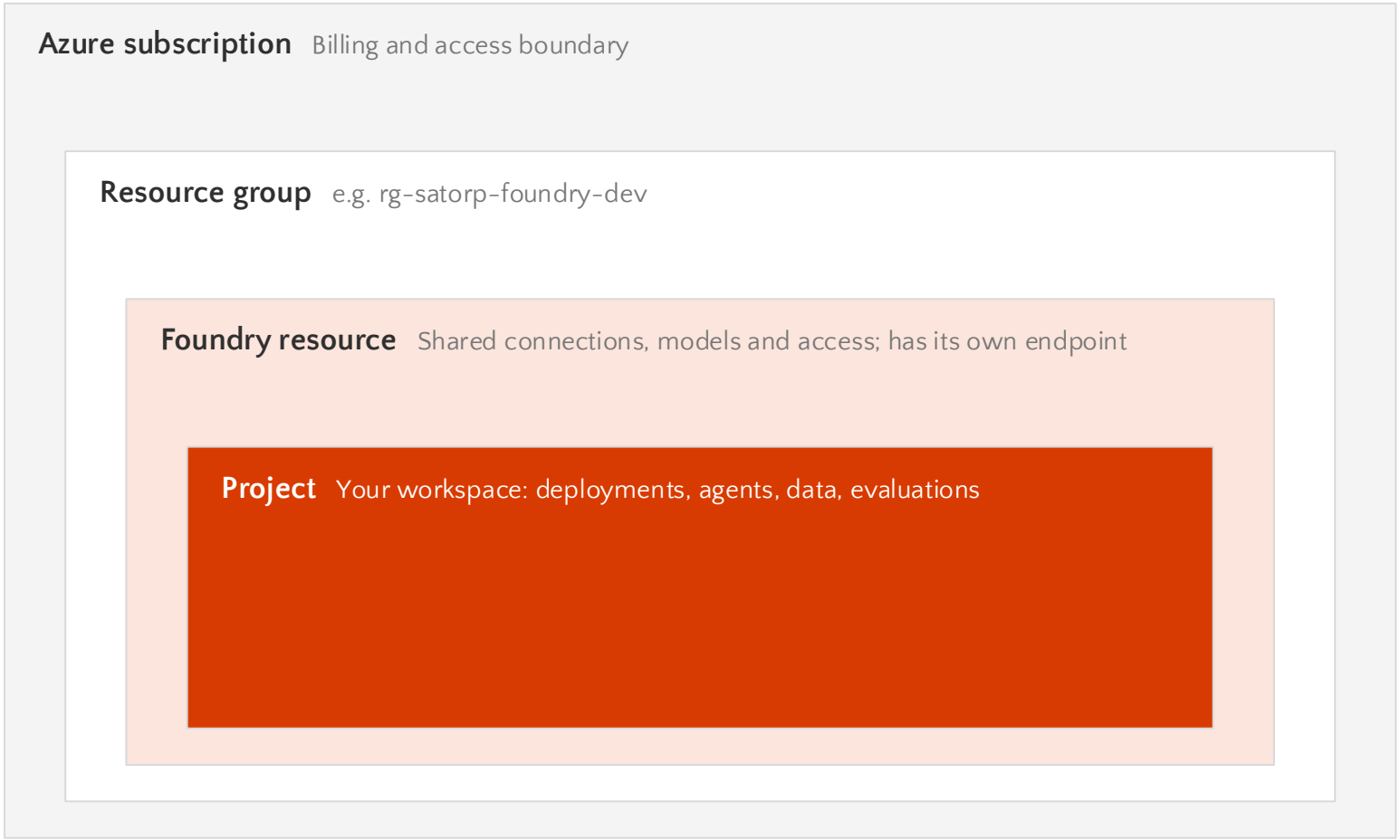
SATORP example

Engineering & turnaround planning agent

- Reads SAP S/4HANA and OpenText privately
- Complex reasoning over P&IDs and documents
- Strict evaluation; owned by a dev team
- Surfaced in Teams through Copilot Studio

Resources and Projects

A Foundry resource in Azure can hold several projects. Projects keep each AI solution's assets together.



LAB 1 SETTINGS

- **Resource:** default name, {project}-resource
- **Subscription:** yours
- **Resource group:** create or select
- **Region:** a Foundry-recommended region

Note your region: you will need it later.

Three Things Your App Needs to Connect

Find them on the project home page after Lab 1.

Item	What it is for	Production guidance
Project endpoint	Models in Foundry through the OpenAI Responses API, and Foundry APIs such as the Agent Service	Use with Entra ID authentication
Azure OpenAI endpoint	Models through OpenAI APIs: Chat Completions and Responses	Use with Entra ID authentication
Key	Key-based authentication to models and tools	Fine for a lab; prefer Entra ID and managed identities in production
Management center	Administer the resource and its projects, users and connections	Assign least-privilege roles

Keys are for labs. Production apps sign in with Entra ID and managed identities.

Working from VS Code: Foundry Toolkit and SDK

Developers spend most of their time in the editor, not the portal.

Foundry Toolkit for VS Code

- **Browse** your Foundry projects and deployments from the sidebar.
- **Try models** in a playground without leaving the editor.
- **Python SDK:** azure-ai-projects with azure-identity.
- **Same code** works locally with your sign-in and in Azure with a managed identity.

Call a deployed model (illustrative Python)

```
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient

project = AIProjectClient(
    endpoint=PROJECT_ENDPOINT,
    credential=DefaultAzureCredential())

client = project.get_openai_client()
response = client.responses.create(
    model="gpt-5-mini",
    instructions="Answer in 3 bullet points.",
    input="What is a P1 work order?")

print(response.output_text)
```

02

Choosing and Comparing Models

Lab 2: explore the catalog, compare on evidence, deploy two models

Model catalog | Model cards | Leaderboard | Trade-offs | Playground

The Model Catalog and Model Cards

A central repository of models, each with a card describing what it is good and bad at.

01 Catalog

Search and filter models by provider, task and capability, e.g. chat, reasoning, embeddings, vision.

02 Model card

What the model does, its limits, its context size and where it is available. Read it before deploying.

03 Benchmarks

How the model compares with similar models on standard quality, safety and performance tests.

04 Deploy

Create a deployment in your project. Your app calls the deployment name, not the raw model.

LAB 2 MODELS

You deploy two models to compare:

- **gpt-5.2:** a larger, more capable model
- **gpt-5-mini:** a smaller, faster, cheaper model

Bigger is not automatically better for your task.

Comparing Models on Four Dimensions

The model leaderboard ranks models and plots trade-offs, so you can see what you gain and what you pay.

01 Quality

How accurate and capable the model is on standard benchmarks.

02 Safety

How well the model avoids harmful or unsafe outputs.

03 Cost

Benchmark cost: what you pay for a comparable amount of work.

04 Throughput

How fast it generates output. Matters for chat users and high volumes.

TRADE-OFF CHART

Plot quality against one other metric.

- Quality vs cost
- Quality vs throughput
- Quality vs safety
- Then Compare models side by side

Shortlist with benchmarks. Decide with your own data.

Big Model or Small Model?

Pick the smallest model that passes your tests. Upgrade only where the evidence says you must.

Consideration	Larger model, e.g. gpt-5.2	Smaller model, e.g. gpt-5-mini
Strengths	Complex reasoning, long multi-step tasks, nuanced writing	Fast, low cost, good at well-grounded, structured tasks
Cost per answer	Higher	Lower: often a fraction of the larger model
Speed	Slower	Faster: better for high-volume chat
Good fit at SATORP	Engineering analysis across many documents	Policy Q&A grounded in SOPs, classification, extraction

Grounding often lets a small model match a large one on answers drawn from your own documents.

Compare Side by Side in the Playground

Same instructions, same question, two models. Look at accuracy, reasoning and style.

Deploy both
gpt-5.2 and gpt-5-mini in your project

→

Set instructions
One system prompt for both, e.g. a travel assistant

→

Compare models
Select the second model in the compare list

→

Ask the same questions
Easy, hard, and ones that should be refused

→

Note differences
Accuracy, reasoning quality, response style

Good for

Quick, hands-on feel for how models differ.

Not enough for

A decision. Ten questions are an anecdote, not evidence.

Next step

Run an evaluation over many questions with consistent scoring.

03

Prompting and Evaluation

Write better prompts, then prove quality with evidence

- Prompt patterns
- Parameters
- Evaluation wizard
- Evaluators
- AI as a judge

The Parts of a Prompt

Models receive messages with different roles. Each one has a job.

01 **System / developer instructions**

Set by the builder: role, rules, tone, output format. Users don't see it.

02 **User message**

The question or task from the person using the app.

03 **Context**

Grounding data added by the app: retrieved documents, tool results, conversation history.

04 **Examples (few-shot)**

A few sample inputs with ideal outputs, so the model copies the pattern.

LAB 2 DEVELOPER PROMPT

You set one instruction for the model under test, e.g.:

- A travel assistant role
- Answer only travel questions
- Refuse unsafe requests

Same prompt for every model you compare.

Prompt Engineering: Before and After

Small changes in a prompt make large changes in quality.

Pattern	Weak prompt	Strong prompt
Give a role	✕ "Answer questions."	✓ "You are a refinery HSE assistant for SATORP staff."
Add context	✕ "What PPE do I need?"	✓ "Using the attached Hot Work SOP, list the PPE required."
Set constraints	✕ (none)	✓ "Use only the SOP. If it isn't there, say you don't know."
Fix the format	✕ (none)	✓ "Reply as a table: Item | Requirement | SOP section."
Show an example	✕ (none)	✓ One sample question with an ideal answer.

Role, context, constraints, format, examples. Then test, change one thing, and test again.

Settings That Change the Answer

Model parameters tune behaviour. Availability varies by model; reasoning models expose fewer of them.

01 Temperature

Lower = more consistent and factual; higher = more varied and creative. Keep it low for policy and data answers.

02 Max output tokens

Caps the length and cost of each answer.

03 Reasoning effort

On reasoning models: how much thinking to spend. More effort = better on hard tasks, slower and costlier.

04 Structured output

Ask for JSON that matches a schema, so your code can read the answer reliably.

DEFAULTS FOR SATORP

- Factual Q&A: low temperature
- Always cap output length
- Structured output for anything a system reads
- Re-evaluate after any change

Parameters are part of the configuration you test.

Why Evaluate? Because Demos Lie

The playground shows what a model can do once. An evaluation shows what it does across many inputs.

An evaluation runs a dataset of inputs through your model or agent and scores every answer the same way.

- **Compare fairly:** two models, same questions, same scoring.
- **Catch regressions:** re-run after every prompt, model or tool change.
- **Prove safety:** include harmful and prompt-injection test cases.
- **Build trust:** show business owners a score, not a feeling.

SATORP PASS MARKS

- ≥ 90% correct on 20–50 real questions
- 0 answers outside approved sources on sensitive topics
- Safety tests pass
- Arabic and English tested where needed

From the SATORP decision guide.

The Evaluation Wizard in Five Steps

From the playground: Evaluations tab → Create.

01 Target	02 Data	03 Configure	04 Criteria	05 Review & submit
What to test Choose Model, then the deployment, e.g. gpt-5.2.	**Test questions** Upload a dataset, or use Synthetic generation from a prompt.	**Developer prompt** Set the instructions the model will use during the run.	**Evaluators** Review suggested evaluators; keep the ones that fit your goal.	**Run it** Name it, e.g. travel-assistant-eval, and submit.

Lab 2 synthetic prompt: “Create various travel related questions, and include some content safety and security tests.”

What Evaluators Measure

Many evaluators use an AI model as a judge, guided by a scoring rubric.

Evaluator	Question it answers	Use it for
Groundedness	Is the answer supported by the provided context?	RAG and knowledge agents
Relevance	Does the answer address the question?	Every assistant
Coherence and fluency	Is it logical, readable and well written?	User-facing answers
Similarity	How close is it to the expected answer?	When you have ground truth
Task adherence, tool-call accuracy	Did the agent follow instructions and call the right tools?	Agents
Safety evaluators	Any harmful content or successful attack?	Everything that goes live

Reading the Results

Scores tell you where to look. The individual answers tell you what to fix.

01 **Start with the summary**

Average score per evaluator, and pass rate against your threshold.

02 **Open the failures**

Read the question, the answer and the judge's reasoning for each low score.

03 **Find the cause**

Wrong source? Weak instructions? Wrong model? Missing tool? Fix one thing.

04 **Re-run and compare**

Run the same dataset again and compare runs side by side.

BEFORE GO-LIVE

- Evaluation set saved and versioned
- Automated in the release pipeline
- AI red teaming run
- Continuous evaluation on live samples

Evaluation is a habit, not a one-off.

04

Agents and Enterprise Security

Running pro-code agents safely: identity, private networking, guardrails, monitoring

Agent Service | SDKs | Security boundaries | SAP reference design | Guardrails

Foundry Agent Service and Friends

The managed runtime for pro-code agents, plus the pieces around it.

Component	What it does
Foundry Agent Service	Managed runtime for agents built on the Responses API. Tools: MCP, Azure AI Search, Fabric data agents, OpenAPI, functions. Supports private networking (bring your own VNet).
Hosted agents (preview)	Run containerised agent code, e.g. Agent Framework or LangGraph, as a managed service.
Microsoft Agent Framework	Open-source SDK for Python and .NET (version 1.0 GA April 2026). MCP, workflows and multi-agent orchestration.
azure-ai-projects SDK	Python and .NET client for projects, deployments, agents and evaluations, used with azure-identity.
Publishing	Expose through your app or API; reach Teams via a Copilot Studio connected agent or the Microsoft 365 Agents SDK.

Four Layers of Protection

The Day 1 Azure foundations, applied to an AI agent.

01 Identity

Users sign in with Entra ID. The agent uses a managed identity, and can pass the user's identity to tools.

02 Access (RBAC)

Least-privilege roles on the project, models, search index and storage.

03 Secrets

No keys in code. Key Vault for anything that must be stored.

04 Network

Private endpoints for Foundry, Search, Storage and Key Vault; public access disabled.

THE GOLDEN RULE

The agent should never see more than the user asking.

- Pass the user's identity to SAP
- Let SAP's own authorisations decide
- Log every call with the user

Security is designed in, not added later.

A Foundry Agent over SAP S/4HANA and OpenText

From the SATORP artifacts: a private, read-only agent that respects SAP authorisations.

Business user	Asks in Teams or a web app; signs in with Entra ID
Foundry Agent Service	Model, instructions and tools, injected into a private Azure VNet
Azure AI Search	Index of OpenText documents for content search (optional)
API Management	Swaps the Entra token for SAP / OpenText tokens; exposes them as MCP tools
ExpressRoute / VPN	Private link to the SAP private cloud: S/4HANA OData APIs and OpenText

Design decisions	
Version 1	Read-only; write-back later with approval
No direct SQL	Always via SAP APIs, never HANA tables
Identity	User's own SAP authorisations apply
Data	No SAP data copied unless indexed
Region	Chosen for data residency
Target	Answers in under 10 seconds

Guardrails: Test It Like an Attacker Would

Phase 6 of the SAP agent guide: prove it is accurate and cannot leak data before any user touches it.

01 Content filters

Azure AI Content Safety on the model deployment blocks harmful content.

02 Prompt Shields

Detects jailbreaks and indirect attacks hidden in documents, e.g. “ignore previous instructions”.

03 Authorisation tests

Same questions as three users with different SAP roles. Each must see only their own data.

04 AI red teaming

Automated and manual attacks: bulk extraction, other vendors’ bank details, injection.

05 Field masking

Mask bank accounts and tax IDs in API Management unless the role needs them.

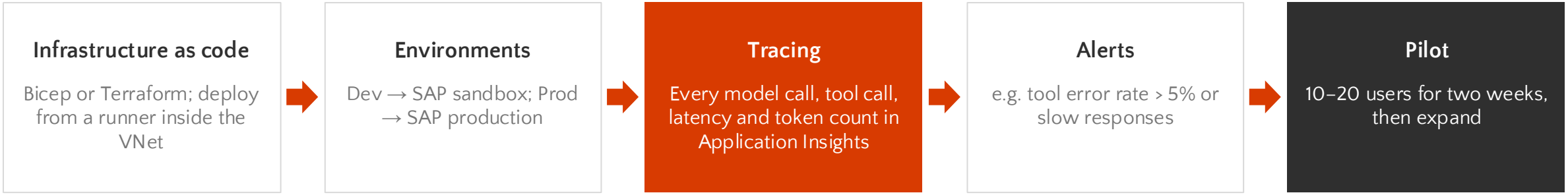
06 Limits

Row limits and field allow-lists in APIM; token budgets per user.

Exit criteria: ≥ 90% accuracy, zero authorisation leaks, all red-team findings closed.

Observability and Operations

Phase 7: deploy from code, watch every run, improve continuously.



Versioning

Every change to instructions or tools is a new agent version; re-run evaluation before promoting.

Cost control

Token budgets per user, right-sized search tier, monthly review.

Runbook

Who to call when SAP is down, certificates expire or firewall rules change.

Today's Hands-on Labs

Both labs run in the Foundry portal at ai.azure.com. Clean up afterwards to stop charges.

01 **Lab 1: Prepare**

~30 min

- Create a Foundry project
- Deploy and test gpt-5.2
- Find endpoints and key
- Install Foundry Toolkit for VS Code

02 **Lab 2: Explore**

Catalog & leaderboard

- Read model cards and benchmarks
- Trade-off charts: cost, throughput, safety
- Deploy gpt-5.2 and gpt-5-mini

03 **Lab 2: Compare**

Playground

- Same prompt, two models
- Note accuracy and style

04 **Lab 2: Evaluate**

Evaluation wizard

- Synthetic dataset
- Quality evaluators
- Review results

Clean up

- Delete the resource group when finished
- Or delete deployments you no longer need
- Check Cost Management the next day

Some features are in preview. If a label differs on your screen, pick the closest match.

What We Covered Today

Foundry is where developers take full control, with evidence and security built in.

Project
resource, endpoints, Entra ID auth

2 models
compared on quality, cost, speed, safety

1 evaluation
synthetic data, AI-judged criteria

1 blueprint
private agent over SAP and OpenText

Remember

- Use Foundry only when a requirement forces code
- Pick the smallest model that passes your tests
- Demos lie; evaluations don't
- The agent never sees more than the user asking

Coming up on Day 4: Data, RAG & Power BI

- Why data quality decides AI quality
- Retrieval-augmented generation end to end
- Azure AI Search: chunking, vectors, hybrid search
- Clean data into a Power BI dashboard

DAY 3 COMPLETE

Thank You. Questions?

Microsoft Data & AI Readiness for SATORP · Day 3: Microsoft Foundry

NEXT

Day 4: Data fundamentals, RAG and Power BI. Please bring Python 3 with pandas, and check access to your Fabric workspace (ws-p1-<your user name>).

www.codesizzler.in